



# Poolora

## How Poolora works

A short note on the system, the tools it uses and how each of them works. B.Tech CSE final year project · Nkosinathi Michael Sibanda · 2026 · launching first in Zimbabwe

### 1. What Poolora is

Poolora connects people who are travelling the same way. A driver publishes a trip with free seats; riders anywhere along that route can book a seat and share the cost. The same idea is extended to **parcels** (send a package with a driver already going there) and **group trips** (plan a holiday together and split the expenses). There are three users: **riders** and **drivers** use the mobile app, and **administrators** use a web dashboard. The first market is **Zimbabwe**: prices in US dollars (ZiG where enabled), payment by EcoCash, OneMoney, InnBucks or card, Harare time, +263 phone numbers. Riders choose a car, SUV or bakkie, minivan, auto (tuk-tuk) or bike.

### 2. The parts of the system

| Part                       | What it does                                                                                                                           | Built with                                  |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| <b>Mobile app</b>          | Screens for riders and drivers: search, booking, payment, live map, chat, SOS, parcels, trips.                                         | React Native, Expo, TypeScript              |
| <b>Backend API</b>         | All business rules: matching, prices, bookings, refunds, safety. Exposes 252 REST routes.                                              | Node.js, Express 5, TypeScript              |
| <b>Real-time gateway</b>   | Pushes live events: the car's position every 5 s, chat messages, SOS alerts and every phone's position during an SOS to admins.        | Socket.IO with a Redis adapter              |
| <b>Car tracker gateway</b> | Reads GPS trackers fitted in drivers' cars and posts their positions to the API; stores nothing. Positions are kept only during rides. | Traccar (forwarder only)                    |
| <b>ML service</b>          | Demand forecast and surge, fraud scoring, and the order to pick up riders.                                                             | Python, FastAPI, NumPy                      |
| <b>Admin dashboard</b>     | Driver verification, SOS desk, disputes, users, fraud flags, payouts, reports, settings.                                               | React, Vite                                 |
| <b>Support assistant</b>   | Answers questions from the help answers and the user's own trips; opens a support request for a person.                                | Claude (Anthropic API)                      |
| <b>Databases</b>           | MongoDB stores the data; Redis caches, rate-limits and locks; Kafka carries events.                                                    | MongoDB, Redis, Kafka                       |
| <b>Outside services</b>    | Sign-in and push; mobile money and card payments; maps, routes and place search; SMS; file storage.                                    | Firebase, Paynow, OpenStreetMap, Twilio, S3 |

The app and the dashboard never talk to the database directly. Every action is an HTTPS request to the API, which checks who the user is, applies the rules, saves to MongoDB and, when needed, sends a live event through Socket.IO or a push notification through Firebase.

### 3. A ride from start to finish

- 1 Sign in.** The user signs in with a phone OTP or Google through Firebase. The app sends the Firebase token to the API, which checks it and returns its own short-lived JWT access token (plus a refresh token).
- 2 Driver publishes a ride.** The API asks OSRM for the driving route, stores it as a GeoJSON line, suggests a fair seat price and checks the rules (at least 2 hours ahead, at most 650 km, seats within the vehicle's limit, price within the allowed range).
- 3 Rider searches.** MongoDB finds rides whose route passes near the rider's pickup and then their drop, in that order. Results are ranked by a match score.
- 4 Rider books and pays.** The API creates a pending booking and a Paynow charge: EcoCash and OneMoney push a PIN prompt to the phone, InnBucks gives a code, cards open Paynow's page (or the money comes from the wallet). Paynow posts the result to the API, which checks its hash; the app also polls, so a lost message never strands a payment.
- 5 Driver accepts.** The seat is reserved in a single atomic database update, so two riders can never get the last seat.
- 6 The ride.** The driver's phone sends its GPS position every 5 seconds over Socket.IO; the rider sees the car move. The driver marks each rider arrived, picked up and dropped off. Safety checks run in the background.
- 7 Settle and rate.** When a rider is dropped, the fare is split into the driver's earnings and the 15% platform fee. The rider gets a receipt and can rate the trip within 7 days.

**Background jobs.** Every minute a job cancels requests left unpaid for 15 minutes, expires requests the driver has not answered in 6 hours, cancels rides nobody booked 1 hour before departure, and sends rating reminders. A Redis lock makes sure only one server runs the job when several are running.

## 4. Key ideas and algorithms

### Matching riders along a route

Each ride stores its route as a GeoJSON LineString with a MongoDB 2dsphere index. A search runs \$geoNear to find routes close to the rider's pickup, keeps those whose line also passes near the drop, then projects both points onto the line. If the pickup comes after the drop, the ride is going the wrong way and is dropped. This finds a rider who joins half-way, which a "near the start point" search would miss.

### Suggested price

$$\text{price} = \text{distance (km)} \times \text{rate for the vehicle} \times 1.1 \text{ at commute hours} \times \text{surge}$$

Rates a seat per km run from US\$0.04 for a bike through US\$0.06 for a sedan to US\$0.075 for an SUV, pitched against kombi and bus fares. Commute hours add 10% (not on public holidays); surge (+20% to +50%) comes from the demand forecast. The driver can move the price 30% either way, but never below US\$0.02 or above US\$0.20 a km, and a seat is at least US\$1. Off-peak examples: a 15 km commute US\$1; Harare to Mutare US\$16; Harare to Bulawayo US\$26.50 (the ordinary bus is US\$15, the luxury coach US\$35).

### Refunds and money

A rider cancelling a confirmed seat gets 100% back 24 hours or more before departure, 50% from 2 hours and nothing after that; cancelling within 30 minutes of the driver accepting is free while the ride is at least 1 hour away. If the driver cancels or moves the time, the rider always gets everything back. Paynow cannot refund, so refunds go to the Poolora wallet, which can be withdrawn to mobile money. Wallet payments only succeed if the balance is enough (checked in

the same database update), every refund has an idempotency key so a retry never pays twice, and group-trip shares are worked out in whole cents so they always add up.

### Settling group expenses

Each member's balance is what they paid minus their share. The person who owes most pays the person owed most, and so on; this greedy method settles everyone with few payments. Example: a US\$300 lodge paid by Tendai and split three ways gives "Rudo pays Tendai US\$100, Farai pays Tendai US\$100". Members pay back through an EcoCash send-money link.

### The ML service

- **Demand forecast:** a weighted formula of hour of day, weekday, past rides, weather and public holidays. The backend sends Harare time and the market's holiday calendar (Easter weekend, Heroes' and Defence Forces Days, Sunday holidays kept on the Monday). It returns a demand level and a surge multiplier, and is deterministic.
- **Fraud scoring:** rules add risk points for many cancellations, unusual payment amounts, many bookings in an hour and new accounts. High risk flags the account; critical risk suspends it until an admin reviews it.
- **Pickup order:** the nearest-neighbour heuristic for the travelling-salesman problem (always visit the closest unvisited pickup next), which runs in  $O(n^2)$ .

### Safety

- **SOS** alerts the admin desk at once and texts emergency contacts a link to the rider's live location.
- **Route deviation:** the car's position is compared with the planned route; more than 500 m off raises an alert.
- **Check-ins:** "Are you OK?" every 30 minutes during a ride; two missed prompts raise an SOS.
- **Trip sharing:** a public link with the car's position that stops working an hour after the trip.
- **Verification:** drivers upload licence, registration book and insurance, and admins check them before the first ride. Accounts are for adults (18+), and a user can close their account from Settings.

## 5. Tools used and how they work

| Tool                       | What it is and how it works                                                                                                                                                                    | Used in Poolora for                                 |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| <b>TypeScript</b>          | JavaScript with types. The compiler checks that values have the right shape before the code runs, then outputs plain JavaScript.                                                               | App, API and admin, so all three share one language |
| <b>React Native + Expo</b> | Write the UI once in React; React Native draws real Android and iOS components. Expo adds ready-made modules (location, notifications) and the build tools.                                    | The mobile app                                      |
| <b>React + Vite</b>        | React builds the page from components that re-render when data changes. Vite is a fast dev server and bundler.                                                                                 | The admin dashboard                                 |
| <b>Node.js + Express</b>   | Node runs JavaScript on the server with one event loop, so it handles many requests while waiting on the database. Express routes each URL to a handler through middleware (auth, validation). | The REST API                                        |
| <b>Socket.IO</b>           | Keeps a WebSocket open between the phone and the server so either side can send a message at any time. Clients join "rooms" (a booking, the admin SOS desk) and the server sends to a room.    | Live GPS, chat, SOS, alerts                         |

| Tool                       | What it is and how it works                                                                                                                                                                               | Used in Poolora for                                     |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| <b>MongoDB + Mongoose</b>  | A document database: each record is a JSON-like document. Indexes make lookups fast; 2dsphere indexes answer questions about points and lines on the Earth. Mongoose adds schemas and validation.         | All stored data, route search                           |
| <b>Redis</b>               | An in-memory key-value store, very fast. Keys can expire on their own. SET with NX gives a simple lock.                                                                                                   | Cache, rate limits, job locks, Socket.IO across servers |
| <b>Apache Kafka</b>        | A log of events that services publish to and read from, so work happens after the request finishes. Poolora falls back to in-process handling when Kafka is down.                                         | Booking, payment and safety events                      |
| <b>Python + FastAPI</b>    | FastAPI turns Python functions into HTTP endpoints and checks the input with Pydantic models.                                                                                                             | The ML service                                          |
| <b>Firebase</b>            | Google's service for sign-in (phone OTP, Google) and push notifications (FCM). The server verifies Firebase tokens with the Admin SDK.                                                                    | Login and push                                          |
| <b>Paynow</b>              | Zimbabwe's payment gateway. The server starts a charge; Paynow prompts the phone (EcoCash, OneMoney), gives an InnBucks code or a card page, then posts the result, signed with a hash the server checks. | EcoCash, OneMoney, InnBucks and card, in US\$ or ZiG    |
| <b>Claude (Anthropic)</b>  | A large language model called through the Anthropic API with tools: it can read the user's recent bookings, quote a cancellation refund and open a support request, but never moves money.                | The in-app support assistant                            |
| <b>OpenStreetMap stack</b> | Free map data. MapLibre draws the map tiles; OSRM computes driving routes; Photon and Nominatim turn text into places.                                                                                    | Maps, routes, place search                              |
| <b>JWT</b>                 | A signed token that carries the user id. The server checks the signature on each request instead of looking up a session.                                                                                 | API authentication                                      |
| <b>Joi</b>                 | Describes what a request body must look like and rejects anything else before it reaches the code.                                                                                                        | Input validation on every endpoint                      |
| <b>Jest, Vitest</b>        | Test runners. Jest with an in-memory MongoDB lets tests use a real database that is thrown away afterwards.                                                                                               | 410 backend, 116 app and 9 admin tests                  |
| <b>Docker, Kubernetes</b>  | Docker packages each service with everything it needs. Kubernetes runs the containers, restarts failed ones and adds copies under load (autoscaler).                                                      | Local stack and deployment                              |
| <b>Git, GitHub Actions</b> | Version control, and CI that runs type checks, lint and tests on every push.                                                                                                                              | Development workflow                                    |

## 6. How it was tested

- Automated tests for the rules that matter most: matching, booking, payments and refunds, settlement, safety, admin actions, parcels and trips. Most run against a real, temporary MongoDB.
- A ride simulator: a bot driver or bot rider lets one person run a complete trip on one phone.
- Manual testing on an Android phone (Samsung Galaxy A04s) and an Android emulator set to Harare, with the backend on a laptop.

## 7. Running it locally

```
npm install (in backend, frontend and admin-web)
backend: npm run dev → API on port 5002
phone: adb reverse tcp:5002 tcp:5002
frontend: npx expo run:android → builds and installs the app
admin-web: npm run dev → dashboard in the browser
```

MongoDB and Redis must be running (Docker Compose starts them). Secrets such as Firebase, Paynow and SMTP keys go in backend/.env; see backend/.env.example and docs/SECRETS.md.